

Leakage-Controlled Concept Graph Construction and Cold-Start Diagnostic Protocol for Knowledge Tracing

Dao Minh Tuan¹, Nguyen Tien Duong¹, Nguyen Khanh Trinh¹, Nguyen Van Hau¹, and Le Hoang Son²

¹ Hung Yen University of Technology and Education, Hung Yen, Vietnam
tuan.ymc@gmail.com

² VNU Information Technology Institute, Vietnam National University,
144 Xuan Thuy Street, Cau Giay Ward, Ha Noi, Vietnam

Abstract. Knowledge tracing (KT) models increasingly use concept graphs, but constructing such graphs from learner interaction logs can introduce leakage when edges are inferred before train/test splitting. We present a leakage-controlled protocol for knowledge component graph construction and auditing in KT. The protocol enforces learner-based temporal splits, train-only edge inference, DAG validation with cycle pruning, cold-start KC stratification, and a DAG Disruption Rate (DDR) diagnostic for graph augmentations. We instantiate the protocol on Junyi Academy, ASSISTments 2012, and XES3G5M, plus two synthetic sanity logs (Synthetic C2/C5) shipped with the repository preprocessing scripts. The audit produces acyclic train-only prerequisite graphs on all benchmarks; the DDR sweep separates four label-agnostic graph augmentation families into four distinct structural regimes: attribute masking leaves prerequisite structure unchanged, edge dropping disrupts it approximately linearly, random-walk subgraph sampling preserves connectivity and disrupts it sub-linearly, and node dropping disrupts it the most through incident-edge cascades. Ground-truth cross-validation on Junyi (the only benchmark in our study that ships with an expert prerequisite annotation) shows that train-only behavioural inference and expert curation are highly correlated at the *node* level (Jaccard ≈ 0.63) but only weakly correlated at the *edge* level (precision/recall F1 ≈ 0.18 at $K=|\text{expert}|$), with low direction agreement (≈ 0.26); we interpret this as evidence that the two sources capture related but distinct aspects of prerequisite structure. Under leakage-controlled evaluation with pyKT checkpoints, GKT does not consistently outperform *simpleKT*/AKT on ASSISTments or XES3G5M (GKT trails by ≈ 0.007 and ≈ 0.039 AUC in the three-fold ASSISTments run and the released XES3G5M fold 0 run, respectively); native GIKT rows are held out pending rerun after fixing an implementation alignment bug, so they are not used for this claim. This observation, surfaced by the protocol rather than by model design alone, motivates broader adoption of leakage-controlled evaluation when comparing graph-augmented and sequence-only KT. Direct leakage metrics ($\text{ECR}_{\text{flag}}=0$ on every corpus) and a train-only versus full-log ablation show headline metric shifts below three decimals on public benchmarks

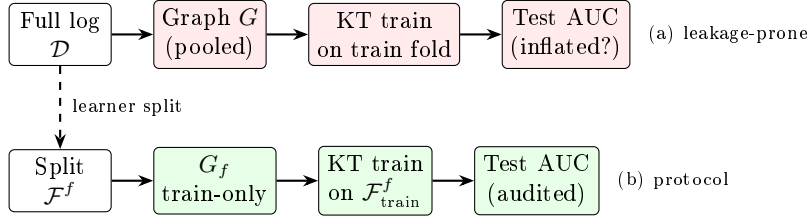


Fig. 1: Illustrative comparison of graph construction workflows. **(a)** Pooling the full log before edge inference lets held-out transitions influence G even when the KT optimiser sees only the train fold. **(b)** Our protocol builds G_f from $\mathcal{F}_{\text{train}}^f$ only and audits provenance before evaluation.

(three-fold ASSISTments ablation: $|\Delta\text{AUC}| < 0.001$). The cold-start diagnostic exposes per-stratum behaviour that aggregate AUC can hide (e.g. Junyi hot vs. warm strata for *simpleKT*). The paper is positioned as a reproducible protocol/resource for graph-enhanced KT, not as a new KT model.

Keywords: Knowledge tracing · Concept graphs · Leakage control · DAG audit · Ground-truth cross-validation · Cold-start diagnostics · Reproducibility

1 Introduction

Knowledge tracing (KT) estimates a learner’s latent mastery of knowledge components from interaction history, supporting adaptive practice, intelligent tutoring, and curriculum recommendation. The field has progressed from Bayesian Knowledge Tracing [9] to deep sequential models [31, 47, 12, 25, 30, 33] and graph-enhanced models that propagate information over relations between knowledge components [28, 36, 44, 7, 10, 41, 6]. These graph-based models rely on auxiliary concept graphs, usually encoding prerequisite or similarity relations.

Problem setting. Let $\mathcal{D} = \{(u, t, q, c, y)\}$ be a KT interaction log with learner u , timestamp t , item q , tagged knowledge component c , and correctness label $y \in \{0, 1\}$. A learner-based temporal split $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ assigns each learner to exactly one fold. For evaluation fold f , the protocol must construct a fold-specific multi-relational concept graph $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using *only* $\mathcal{F}_{\text{train}}^f$, then train a KT model on $\mathcal{F}_{\text{train}}^f$ and score predictions on $\mathcal{F}_{\text{test}}^f$. The central question is whether reported graph-augmented KT gains remain valid when G_f is audited for leakage, acyclicity, and cold-start coverage.

Figure 1 contrasts the two workflows. In workflow (a), prerequisite or similarity edges may be supported by validation/test interactions; the KT module then appears to generalise while part of the structural input already encodes future behaviour. Workflow (b) closes this channel by making graph construction a first-class, fold-conditioned step.

Real-world significance. Leakage-controlled concept graphs matter wherever KT systems drive high-stakes decisions. Intelligent tutoring platforms use mastery estimates to schedule practice and gate advanced modules; incorrect graphs can push learners into topics for which prerequisites were never mastered [15,18]. Curriculum analytics teams compare cohorts across terms; graphs built from pooled logs can confound term effects with structural shortcuts. MOOC and corporate training pipelines increasingly deploy graph-augmented KT [8,27]; without fold-specific audits, reported AUC improvements are difficult to attribute to modelling versus preprocessing. Our protocol targets these deployment settings by making graph provenance, DAG validity, and cold-start strata explicit reporting artefacts rather than hidden preprocessing choices.

A key risk is that concept graphs are often inferred from the same interaction logs used to train and evaluate KT models. If graph edges are computed from the full log before train/test splitting, held-out information can leak back into the model through the graph. This leakage may occur even when the KT model itself is trained only on the training fold. Benchmarking tools such as `pyKT` [26] have highlighted the importance of preprocessing and split discipline, but a fold-specific audit protocol for KC graph construction remains under-specified.

A second risk arises when generic graph augmentations are imported into KT. Node dropping, edge dropping, attribute masking, and subgraph sampling were originally designed for general-purpose graphs [45,49,42,35,37,14]. However, prerequisite graphs are directional and educationally meaningful: removing a prerequisite edge may remove a pedagogical constraint. The degree to which label-agnostic graph augmentations disturb educational prerequisite DAGs has not been systematically measured in KT.

A third issue is the informal use of “cold-start” in KT. In this paper, we define cold-start at the KC level, based on the number of train-fold interactions associated with each KC, in contrast to user- or item-level cold-start in recommender systems [32]. This separates cold-start KC behaviour from other cold-start settings.

We address these issues at the level of protocol rather than model. We do not propose a new KT architecture or claim state-of-the-art performance. Instead, we release a leakage-controlled graph construction and audit protocol, a structural augmentation diagnostic, a ground-truth cross-validation procedure (instantiated on Junyi), and a cold-start KC report. The protocol is instantiated on Junyi Academy [18,3], ASSISTments 2012, and XES3G5M [27].

Novelty. Relative to prior KT surveys and benchmarks [1,26], our contribution is not a new predictive backbone but a *reproducible audit layer* around concept-graph construction. Compared with graph-KT papers that treat G as a fixed input [28,36,44,7,10], we (i) formalise fold-specific graph construction with explicit leakage diagnostics, (ii) introduce DDR to quantify how generic augmentations disrupt prerequisite DAGs, (iii) operationalise KC-level cold-start reporting, and (iv) ground behavioural prerequisite inference against expert curation where available. Compared with prerequisite-learning surveys [2,23,29], we do not propose a new inference rule; instead we audit a fixed train-only

rule and quantify disagreement with expert graphs. Compared with graph contrastive KT [22,34,38], we separate *structural* disruption of educational DAGs from downstream AUC, which can mask graph-quality issues.

Contributions.

- **C1.** A train-only protocol for constructing and auditing multi-relational KC graphs $G = (\mathcal{C}, E_{\text{pre}} \cup E_{\text{sim}})$ for KT, including split checks, edge provenance, and DAG validation with cycle pruning.
- **C2.** The DDR metric, which quantifies how generic graph augmentations disrupt prerequisite assumptions on educational DAGs, instantiated with four formally defined augmentation operators.
- **C3.** A cold-start KC diagnostic with explicit frequency-based strata and reproducible scripts on three KT benchmarks.
- **C4.** A ground-truth cross-validation procedure that quantifies, on Junyi Academy, how train-only behavioural inference of E_{pre} agrees with the expert prerequisite annotation of Chang et al. [18], using edge-level, direction-level, and reachability-level metrics across a sweep of top- K cutoffs.
- **C5.** An empirical observation, surfaced by the protocol on ASSISTments (three-fold learner CV) and XES3G5M pyKT checkpoints, that GKT does not consistently outperform *simpleKT*/AKT once train-only graph construction is enforced, together with a train-only versus full-log ablation and direct leakage metrics. We discuss three non-mutually-exclusive hypotheses for this pattern and frame it as a motivation for leakage-controlled evaluation as a benchmarking precondition rather than as a refutation of graph-augmented KT.

Scope. Tables 7–11 report diagnostic pyKT checkpoints with train-only exported graphs (valid+test sequence positions). ASSISTments 2012 uses three-fold learner CV with 95% bootstrap confidence intervals; other corpora use the released single-fold diagnostic run, where bootstrap intervals collapse to the point estimate when `n_folds=1`. Protocol metrics (DDR, leakage tables, DAG audit over folds 0–2) use the full three-fold configuration unless noted. Junyi omits most graph-augmented pyKT rows in the main baseline table (GKT, GIKT, DyGKT, DGEKT) because full training exceeded the released memory budget; native SKT is included from the fold 0 cache where available. Graph ablation on Junyi still reports the graph-model family where completed. DAG audit and ground-truth cross-validation prose refer to fold 0 unless noted. We do not claim that the protocol eliminates every possible form of leakage, nor do we make claims about real-world learning outcomes.

2 Related Work

2.1 Knowledge Tracing and Benchmark Discipline

Early KT relied on probabilistic latent-state models such as Bayesian Knowledge Tracing (BKT) [9] and extensions with individualised parameters [46,19]. Deep

KT introduced recurrent sequence models (DKT) [31], memory-augmented variants (DKVMN) [47], and attention-based architectures (AKT, simpleKT) [12,25,5,39]. Self-attentive and convolutional designs (RKT, SAINT) [30,33] further improved sequence modelling. Large-scale surveys [1] document rapid architectural progress but also heterogeneous preprocessing. The pyKT library [26] standardises training loops and metrics across datasets including EdNet [8] and XES3G5M [27].

Limitations of this line for our setting. Most KT benchmarks treat the interaction log as the sole input and optimise predictive metrics (AUC, ACC, NLL) without auditing auxiliary structure. Preprocessing choices—sequence length, Q-matrix construction, split type—can shift rankings by several points [26,1], yet graph provenance is rarely logged. Aggregate metrics also hide per-knowledge component behaviour: a model can appear strong overall while failing on rare skills. Our protocol complements benchmark libraries by standardising fold-specific concept graphs and KC-level cold-start reporting rather than proposing another sequence backbone.

2.2 Graph-based Knowledge Tracing and Prerequisite Structure

GKT [28] first propagated mastery signals over a fixed KC graph with graph neural networks. SKT [36] separates prerequisite and similarity relations; GIKT [44] adds question–KC structure. Follow-ups include hierarchical and interaction-rich designs (HGKT, GrapKT, IEKT) [6,41,24], dynamic graphs (DyGKT) [7], dual-graph ensembles (DGEKT) [10], and context-aware graph extensions [43,16]. In parallel, prerequisite-learning research infers directed relations from MOOC logs, content, or expert crowdsourcing [23,29,4,21,2,3].

Limitations of this line for our setting. Graph-KT papers typically *assume* a concept graph is given and optimise predictive loss, leaving graph construction outside the experimental factorisation. When graphs are inferred from the same log used for evaluation, held-out transitions can influence edges—a channel distinct from label leakage in the KT loss. Prerequisite-learning work optimises edge quality against expert or content signals but seldom couples inference to KT splits or reports DAG audit trails. Expert annotations (e.g. Junyi [3,18]) and behavioural inference capture related but non-identical structure; treating one as a proxy for the other is empirically unsupported without cross-validation. Our protocol keeps inference fixed and auditable, then quantifies disagreement with expert graphs where available.

2.3 Graph Augmentation, Contrastive KT, and Reproducible Resources

General graph self-supervised learning popularised label-agnostic augmentations: GraphCL [45], GCA [49], SimGRACE [42], BGRL [35], DGI [37], and multi-view contrastive methods [14,48,11]. GNN encoders (GCN, GraphSAGE) [20,13] underpin many KT graph modules. These augmentations have been imported into contrastive KT (CL4KT, Bi-CLKT, DECKT) [22,34,40,38], usually optimising representation quality on interaction sequences or bipartite graphs.

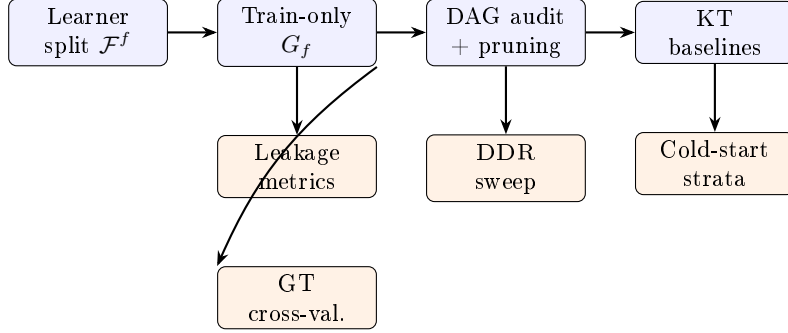


Fig 2: Research framework connecting protocol modules (Section 3) and experiment tracks (Section 4). **Strengths:** fold-conditioned provenance, explicit DAG repair, structural diagnostics decoupled from AUC, optional expert cross-validation. **Limitations:** fixed behavioural inference rule; similarity rule fails on single-skill Q-matrices; DDR covers only four generic augmentations; GT CV available only where expert graphs exist.

Limitations of this line for our setting. Augmentation papers optimise invariance objectives on general graphs; educational prerequisite DAGs impose directionality and pedagogical interpretability that random edge or node drops can violate. Empirical studies on general graphs report task-dependent augmentation rankings [48], but DDR effects on prerequisite DAGs are not part of standard KT reporting. Contrastive KT work rarely reports whether augmentations preserve acyclicity or alter reachability between knowledge components. Resource-oriented reproducibility in KT has focused on code and splits [26] rather than on graph audit logs. We release provenance artefacts, DDR sweeps, and cold-start tables so graph-enhanced KT studies can separate structural disruption from predictive gains.

Positioning. Our work is orthogonal to inventing new KT architectures or prerequisite inference rules. It supplies an audit layer—leakage diagnostics, DAG pruning logs, DDR, ground-truth cross-validation, and cold-start strata—that existing graph-KT and contrastive-KT papers typically omit.

3 Protocol

Let $\mathcal{D}$ be a KT interaction log, $\mathcal{C}$ the set of knowledge components, and $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ a learner-based temporal split. The protocol constructs a fold-specific graph $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using only $\mathcal{F}_{\text{train}}^f$.

Figure 2 summarises module dependencies. Split discipline is a hard precondition: without learner-disjoint folds, downstream diagnostics are not interpretable. Graph construction and DAG audit are coupled: train-only inference can introduce cycles, so pruning must be logged before any graph-augmented KT run. Leakage metrics, DDR, cold-start strata, and ground-truth cross-validation are

parallel audits—they do not retrain models—which keeps the protocol light weight but means it cannot certify every possible leakage path (e.g. hidden features in proprietary pipelines).

3.1 Formal Problem Statement and Leakage Audit

For each evaluation fold f , the protocol seeks a graph $G_f = (\mathcal{C}, \mathcal{E}_f)$ with $\mathcal{E}_f = E_{\text{pre}}^f \cup E_{\text{sim}}^f$ that is *feasible* with respect to train-only evidence and structural constraints, then verifies low direct leakage on scalar diagnostics. Let $\mathcal{I}_{\text{tr}}^f$ denote train-fold interactions and $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ held-out interactions.

Decision variables. Edge sets $E_{\text{pre}}^f \subseteq \mathcal{C} \times \mathcal{C}$ and $E_{\text{sim}}^f \subseteq \mathcal{C} \times \mathcal{C}$, edge weights $w_{ij} \geq 0$, and pruning decisions on cyclic subsets of E_{pre}^f .

Objective. The construction stage maximises retained pedagogical structure subject to sparsity and acyclicity:

$$\max_{E_{\text{pre}}^f, E_{\text{sim}}^f} \sum_{(i,j) \in E_{\text{pre}}^f} w_{ij} + \lambda \sum_{(i,j) \in E_{\text{sim}}^f} \text{sim}(c_i, c_j) \quad (1)$$

where sim is Jaccard or PMI (Section 3.4) and $\lambda \geq 0$ trades prerequisite mass against similarity mass (in practice $\lambda=0$ because the two edge types are inferred in separate pipelines).

Constraints.

$$(C1) \forall e \in \mathcal{E}_f, S_{\text{hi}}(e) = \emptyset; \quad (2)$$

$$(C2) \mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}} = \emptyset,$$

$$\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}} = \emptyset,$$

$$\mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}} = \emptyset; \quad (3)$$

$$(C3) E_{\text{pre}}^f \text{ is acyclic after pruning}; \quad (4)$$

$$(C4) |E_{\text{pre}}^f| \leq K, |\text{Out}_{E_{\text{pre}}^f}(i)| \leq k;$$

$$\text{sim}(c_i, c_j) \geq \tau \forall (i, j) \in E_{\text{sim}}^f; \quad (5)$$

$$(C5) \text{TBVR}_f = 0; \quad (\text{no post-cutoff train transitions in edge evidence}). \quad (6)$$

C1–C2 block cross-fold and cross-learner contamination; C3–C4 encode the DAG audit and top- K /top- k filters of Section 3.4; C5 is checked by Eq. (10).

Audit functionals (diagnostics, not training losses). Because C1 cannot always be verified from AUC alone, the protocol reports four scalar functionals on G_f : $\text{ECR}_f^{\text{flag}}$ (Eq. (7)), $\text{ECR}_f^{\text{overlap}}$ (Eq. (8)), EOC_f (Eq. (9)), and TBVR_f (Eq. (10)). Ideal hygiene: $\text{ECR}_f^{\text{flag}}=0$, $\text{TBVR}_f=0$, modest EOC_f ; large $\text{ECR}_f^{\text{overlap}}$ may still occur under honest splits when curricula repeat across disjoint learners.

Table 1: Taxonomy of leakage classes audited by the protocol: typical source of contamination, a concrete KT example, and the primary direct diagnostic used in this paper (Section 3.2).

Class	Source	Example	Metric
Structural	Learner identifiers reused across validation, and test folds	The same learner contributes interactions to both train and held-out partitions under fold f	ECR^{flag}
Edge	Interactions outside $\mathcal{F}_{\text{train}}^f$ used as evidence for edges	Prerequisite or similarity edge retained because val/test transitions inflate co-occurrence or transition counts	$\text{ECR}^{\text{overlap}}$
Target	Outcomes or mappings pooled across splits	Q-matrix or calibration statistics fit on all learners; edge weights implicitly tuned against aggregated correctness	EOC
Temporal	Violating the timestamp boundary τ_u per learner u	Post-boundary attempts included when counting $c_t \rightarrow c_{t+1}$ support or when building time-sensitive features	TBVR
Cold-start neighbourhood	Tail-knowledge component edges dominated by non-train support	Rare-knowledge component edge driven mainly by interactions that belong to val/test under the split	$\text{ECR}^{\text{overlap}} (+ \text{strata})$

3.2 Leakage Control

We audit five leakage classes—*structural*, *edge*, *target*, *temporal*, and *cold-start neighbourhood*—summarised in Table 1. Structural leakage occurs when the same learner contributes interactions to more than one of train, validation, or test under a fold, coupling splits at the identifier level. Edge leakage occurs when similarity or prerequisite edges receive statistical support from interactions outside the train fold of that evaluation split. Target leakage occurs when correctness labels, outcome-conditioned statistics, or other supervision-derived artefacts influence graph construction or features after pooling across splits (including fitting item-KC mappings on full data instead of expert tags or train-only evidence). Temporal leakage occurs when interactions at or after the learner-specific split boundary contribute to edge inference or to graph-derived inputs at evaluation time. Cold-start neighbourhood leakage is the special case where edges touching rare knowledge components are justified primarily by held-out or future-split interactions, so tail-knowledge component structure mirrors aggregate performance rather than train-plausible evidence.

Each inferred edge is logged with its fold of origin, timestamp range, relation type, and train-only flag; severe violations halt the pipeline. In addition, we

report four scalar *direct* leakage diagnostics that do not reduce to predictive accuracy. Let $\mathcal{U}_f^{\text{tr}}$, $\mathcal{U}_f^{\text{va}}$, and $\mathcal{U}_f^{\text{te}}$ denote the learner-id sets in train, validation, and test under fold f . The learner overlap flag is

$$\text{ECR}_f^{\text{flag}} := \mathbb{1}[(\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}}) \cup (\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}}) \cup (\mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}}) \neq \emptyset]. \quad (7)$$

When folds are learner-disjoint, $\text{ECR}_f^{\text{flag}} = 0$ for every fold—the regime enforced below.

Let $\mathcal{E}_f$ denote the graph edge set (prerequisite and similarity) consumed when evaluating fold f , and let $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ denote held-out interactions under that split. For an edge $e = (i, j)$, let $S_{\text{tr}}(e)$ be the multiset of train-fold interaction tokens counted toward e 's evidence, and let $S_{\text{hi}}(e)$ be the analogous multiset restricted to $\mathcal{H}_f$. Define the overlap indicator $\chi_{\text{ECR}^{\text{overlap}}}(e) = \mathbb{1}[|S_{\text{hi}}(e)| > 0]$. The held-out pattern overlap rate mirrors an edge-level importance scan (cf. feature importance plots): it asks whether each retained edge aligns with *some* mass observable among held-out interactions—possibly contributed by disjoint learners—rather than encoding learner pooling across folds,

$$\text{ECR}_f^{\text{overlap}} = \frac{1}{|\mathcal{E}_f|} \sum_{e \in \mathcal{E}_f} \chi_{\text{ECR}^{\text{overlap}}}(e). \quad (8)$$

For Edge–Outcome Correlation, stack normalized edge weights into $\mathbf{w}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ (for ties, use the order exported by the audit log). Let $\mathbf{y}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ carry test-fold outcome summaries aligned to each edge—for example, endpoint averages of correctness on $\mathcal{F}_{\text{test}}^f$ or their logits—computed *after* graph construction (labels are used only for this diagnostic, not for inferring $\mathcal{E}_f$). Let $R_f \in [-1, 1]^{2 \times 2}$ be the Pearson correlation matrix between the two scalar sequences (w, y) pooled over edges with defined variance.

$$\text{EOC}_f = \|R_f\|_F, \quad (9)$$

For binary correctness summaries this matches the spirit of a 2×2 correlation-matrix summary relating structural edge mass to outcome mass; if $\text{Corr}(\mathbf{w}_f, \mathbf{y}_f) = \rho$, then R_f has unit diagonal and off-diagonals ρ , hence $\text{EOC}_f = \sqrt{2 + 2\rho^2}$.

Let $t_{u,i}$ be the timestamp of interaction i for learner u and let τ_u be u 's temporal split cutoff for fold f . Write $\mathcal{I}_{u,f}^> = \{i : t_{u,i} > \tau_u\}$ for post-boundary interactions and $\mathcal{V}_{u,f}$ for the subset of those tokens that incorrectly appear in any multiset counted toward $\mathcal{E}_f$ or in graph-derived features at inference for fold f . The Temporal Boundary Violation Rate is

$$\text{TBVR}_f = \frac{\sum_u |\mathcal{V}_{u,f}|}{\sum_u |\mathcal{I}_{u,f}^>|}. \quad (10)$$

Under learner-disjoint splits, no learner spans train and held-out folds; we therefore instantiate τ_u *within* $\mathcal{F}_{\text{train}}^f$ by taking τ_u at the index that separates the earliest $\lfloor rn_u \rfloor$ chronologically ordered train interactions from the tail of learner u , where r matches the protocol train ratio (here $r=0.7$) and n_u is u 's train-fold

length. The multiset checked is the transition evidence underlying the retained prerequisite edges E_{pre}^f . Ideal structural hygiene yields $\text{ECR}_f^{\text{flag}}=0$, temporal hygiene yields $\text{TBVR}_f=0$, and we additionally desire a modest EOC_f relative to permuted-reference baselines. The overlap statistic $\text{ECR}_f^{\text{overlap}}$ *need not* vanish under honest splits because ubiquitous curricula reproduce frequent prerequisite transitions across disjoint learners; large $\text{ECR}_f^{\text{overlap}}$ therefore must not be read as $\text{ECR}_f^{\text{flag}}=1$ or as a train-only provenance violation without consulting the audit trail.

Why AUC/ACC are insufficient. High test AUC or accuracy does *not* certify absence of leakage: held-out information can be *baked into* the graph (or static artefacts) before optimisation, so the model merely exploits a consistent shortcut rather than measuring generalisation under honest provenance. Conversely, scrubbing leakage can lower headline metrics without making the model worse in substance—it removes illegitimate signals. Direct diagnostics such as $\text{ECR}_f^{\text{flag}}$, $\text{ECR}_f^{\text{overlap}}$, EOC_f , and TBVR_f therefore complement discrimination scores by testing whether the structural inputs themselves remain statistically independent of the wrong folds or outcomes.

3.3 Graph Representation for Knowledge Tracing

We represent each fold’s knowledge structure as a multi-relational directed graph $G_f = (\mathcal{C}, \mathcal{R}, \mathcal{E}_f)$ where nodes $c \in \mathcal{C}$ are knowledge components (hashed identifiers in the released pipeline), and edges carry relation type $r \in \mathcal{R} = \{\text{pre}, \text{sim}\}$. Prerequisite edges $(i, j) \in E_{\text{pre}}^f$ encode directed precedence inferred from temporal transitions; similarity edges $(i, j) \in E_{\text{sim}}^f$ encode co-assessment structure from the train-fold Q-matrix. Each edge stores provenance fields (fold id, train-only flag, source tag, support count, weight w_{ij}) exported as CSV for audit.

For graph-augmented KT baselines, G_f is materialised as a row-normalised adjacency matrix $A_f \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{C}|}$ (`pykt_graph_matrix.py`): prerequisite and similarity relations can be merged or consumed separately depending on the backbone (GKT uses a single KC graph; GIKT additionally uses question–KC bipartite structure from the train fold). Node features are not learned at graph-construction time; the protocol treats G_f as a *structural prior* consumed by downstream KT optimisers. This separation mirrors GNN practice on citation or knowledge graphs [20,13] but with education-specific relation semantics and fold conditioning.

Technology stack: graph construction is NumPy/pandas/NetworkX; cycle detection uses Johnson’s algorithm [17]; KT training optionally delegates to pyKT [26] with exported NPZ adjacency. All artefacts are versioned per dataset and fold under `data/graphs/<dataset>/fold_<f>/`.

3.4 Train-only Graph Construction

Prerequisite edges E_{pre} . For uniformity across heterogeneous benchmarks, we infer E_{pre} from train-only temporal transitions on *all* three datasets, independently

of whether an external prerequisite source is also available for that dataset. This deliberate choice decouples the audit pipeline from dataset-specific quality of expert annotations and ensures that every reported edge originates from a single, auditable inference rule. When an expert annotation is available for a benchmark (here, Junyi Academy), the protocol additionally performs ground-truth cross-validation as a separate audit step (Section 4.7).

Concretely, for each train-fold interaction sequence we extract consecutive KC transitions $c_t \rightarrow c_{t+1}$ with $c_t \neq c_{t+1}$, count their support $s(c_i, c_j)$, and retain candidate edges in three filtering stages: (i) a support-quantile filter that drops transitions below the q -th percentile (default $q = 0.95$); (ii) a per-source top- k cap (default $k = 5$) that retains, for each source KC, only the k destination knowledge components with highest support; and (iii) a global top- K cap on the number of retained candidate edges (default $K = 5,000$). Each retained edge carries the normalized support weight $w_{ij} = s(c_i, c_j)/s_{\max}$ used by the cycle pruning step in Section 3.5. This candidate edge set is then passed unchanged to the DAG audit; cycle pruning operates only on the candidates returned by stages (i)–(iii).

Similarity edges E_{sim} and theoretical properties. We compute E_{sim} from a train-only Q-matrix using either Jaccard or PMI co-occurrence at the item level (default Jaccard with threshold $\tau=0.10$). Let $I(c) \subseteq \mathcal{Q}$ be the train-fold item set tagged with concept c , with $n = |\mathcal{Q}|$ items:

$$\text{sim}_J(c_i, c_j) = \frac{|I(c_i) \cap I(c_j)|}{|I(c_i) \cup I(c_j)|}, \quad (11)$$

$$\begin{aligned} \text{sim}_{\text{PMI}}(c_i, c_j) &= \log \frac{p(c_i, c_j)}{p(c_i)p(c_j)}, \\ p(c) &= \frac{|I(c)|}{n}, \\ p(c_i, c_j) &= \frac{|I(c_i) \cap I(c_j)|}{n}. \end{aligned} \quad (12)$$

Properties. (i) *Boundedness:* $0 \leq \text{sim}_J \leq 1$; edges are retained iff $\text{sim}_J \geq \tau$. (ii) *Symmetry:* both scores are symmetric in (c_i, c_j) ; we store directed permutations (i, j) and (j, i) for pipeline uniformity. (iii) *Empty overlap:* if items are single-skill, $|I(c_i) \cap I(c_j)| = 0$ for $i \neq j$ and both metrics yield no edges—the ASSISTments 2012 case in Section 4.4. (iv) *PMI dominance:* when $|I(c_i) \cap I(c_j)|$ is small but non-zero, PMI can rank rare co-tagged pairs above Jaccard because it reweights by marginal frequencies; threshold τ on raw PMI therefore requires dataset-specific calibration unlike Jaccard’s scale-free $[0, 1]$ range. (v) *Monotonicity in co-tagging:* holding marginals fixed, increasing overlap never decreases sim_J ; PMI can decrease if marginals grow faster than co-occurrence.

As discussed in Section 4.1, this similarity rule is sensitive to the multi-skill density of the underlying Q-matrix.

Subgraph construction for KT consumption. Beyond full G_f , some KT pipelines and augmentation operators consume *induced subgraphs*. Given visited node set

$S \subseteq \mathcal{C}$, the induced prerequisite subgraph is $G_f[S] = (S, E_{\text{pre}}^f \cap (S \times S))$. The random-walk subgraph operator in Section 3.6 samples S with $|S| \geq \lceil (1 - p)|\mathcal{C}| \rceil$ by a walk on the undirected skeleton of E_{pre}^f , preserving internal edges while dropping boundary incident edges—hence sub-linear DDR growth. For KT matrix export, we optionally restrict to the largest weakly connected component of E_{pre}^f when isolates would otherwise yield zero row mass; this is logged but does not change edge provenance. The train-only versus full-log ablation (Section 4.6) contrasts two global graphs G^{train} and G^{full} built from $\mathcal{F}_{\text{train}}^f$ versus the entire preprocessed log; subgraph operators then act on whichever parent graph is supplied.

All graph artefacts are materialised per dataset and fold so that downstream models cannot accidentally consume full-data edges.

3.5 DAG Audit

For each fold-specific prerequisite edge set, we check whether E_{pre} is acyclic. If cycles are detected, we iteratively remove the lowest-confidence edge in a detected cycle and re-run the check until a topological ordering exists. Cycle detection uses a bounded simple-cycle enumeration procedure based on Johnson’s algorithm for enumerating elementary circuits in directed graphs [17], capped at 100 representative cycles per detection round. We use this cap because the number of elementary cycles can grow rapidly with graph density; exhaustive enumeration is unnecessary for our audit, since the pruning loop only requires representative cycle witnesses. The detector cap does not affect correctness: the pruning loop terminates only when no further cycle can be detected, so the final retained graph is acyclic. Each removed edge is written to `results/reports/<dataset>_dag_pruning_log.csv` with its source, target, confidence score, and pruning reason. The audit reports node count, raw edge count, pruned edge count, retained edge count, root count, leaf count, and final topological-sort status. The claim is not that the retained DAG is the unique correct prerequisite hierarchy, but that the pruning rule and edge provenance are explicit and reproducible.

3.6 DAG Disruption Rate and Augmentation Operators

To quantify the structural effect of graph augmentations on prerequisite DAGs, we define the DAG Disruption Rate (DDR). Let A be an augmentation operator and $A(E_{\text{pre}}, p)$ the edge set obtained at perturbation strength p . Let $\text{pre}(A(E_{\text{pre}}, p)) \subseteq E_{\text{pre}}$ be the original prerequisite edges that remain with their original direction, and let $\text{rev}(A(E_{\text{pre}}, p))$ be the edges that appear with the opposite direction. We define

$$\text{DDR}(A, p) = \frac{|E_{\text{pre}} \setminus \text{pre}(A(E_{\text{pre}}, p))| + |\text{rev}(A(E_{\text{pre}}, p))|}{|E_{\text{pre}}|}. \quad (13)$$

We evaluate four label-agnostic augmentation families. Their implementations follow GraphCL [45] and are applied to the retained acyclic E_{pre} :

- **Attribute masking.** Random subset of edge attributes is masked (set to a neutral value). The edge set itself is unchanged, so attribute masking induces $\text{DDR} = 0$ by construction.
- **Edge dropping.** Each prerequisite edge is independently removed with probability p . Expected DDR is approximately p , modulo finite-sample variation.
- **Subgraph sampling (random walk).** Starting from a uniformly random seed node, a random walk over the (undirected) prerequisite graph extends the visited node set until it reaches $\lceil (1-p)|\mathcal{C}| \rceil$ nodes; if the walk reaches a dead end before the target size, it restarts from a random unvisited node. The induced edge subgraph on the visited nodes is returned. Connectivity within the visited set is preserved, so disruption comes only from edges incident to non-visited nodes.
- **Node dropping.** A random subset of $\lceil p|\mathcal{C}| \rceil$ nodes is removed, together with all their incident edges. Disruption is amplified by the incident-edge cascade.

Each operator is swept over $p \in \{0.05, 0.10, 0.20, 0.30\}$ with three random seeds on each of three train folds ($3 \times 3 = 9$ runs per setting); reported means aggregate across those runs. DDR is a structural diagnostic only: it measures how much prerequisite structure is removed or reversed by an augmentation, not whether a downstream KT model becomes better or worse.

3.7 Cold-start KC Stratification

We define cold-start at the KC level, distinct from user/item cold-start in recommender systems [32]. For fold f , let $n_f(c) = |\{(u, t, q, c, y) \in \mathcal{F}_{\text{train}}^f\}|$ be the number of train-fold interactions tagged with knowledge component c . Each KC is assigned to one of four frequency strata:

$$\begin{aligned}
 \mathcal{S}_1 &: n_f(c) < 20, \\
 \mathcal{S}_2 &: 20 \leq n_f(c) < 100, \\
 \mathcal{S}_3 &: 100 \leq n_f(c) < 500, \\
 \mathcal{S}_4 &: n_f(c) \geq 500.
 \end{aligned}$$

Technical procedure. (i) *Stratum assignment* is computed after the learner split and uses only $\mathcal{F}_{\text{train}}^f$ counts (no test labels). (ii) *Interaction tagging*: each test interaction inherits the stratum of its knowledge component tag; interactions whose knowledge component never appears in train are flagged `out_of_range` and excluded from stratum means. (iii) *Metric aggregation*: for each model m and non-empty stratum $\mathcal{S}_k$, we compute AUC, ACC, and NLL on the subset of test rows with $c \in \mathcal{S}_k$, using the same prediction export as the aggregate baseline table. (iv) *Neighbourhood leakage link*: if a rare knowledge component $c \in \mathcal{S}_1$ has similarity or prerequisite edges supported mainly by held-out mass, $\text{ECR}_f^{\text{overlap}}$ rises (Table 1); cold-start strata make such failures visible when aggregate AUC looks stable. (v) *Reporting*: results are written to

results/tables/cold_start_by_stratum.csv with columns `dataset`, `model`, `stratum`, `auc`, `acc`, `nll`, `n_interactions`.

This prevents aggregate metrics from hiding behaviour on rare knowledge components and supports fair comparison between sequence-only and graph-augmented models on tail skills where graph smoothness is often hypothesised to help.

3.8 Ground-Truth Cross-Validation

When a benchmark provides an expert prerequisite annotation, the protocol performs an additional, optional audit step: comparing the train-only inferred E_{pre} against the expert edge set $E_{\text{pre}}^{\text{expert}}$. The procedure has four stages: (i) load the expert annotation and filter rows by a prerequisite-score threshold θ to obtain a directed expert edge set; (ii) align expert KC identifiers to the pipeline’s hashed `kc_id` space via a deterministic name-to-id mapping reconstructed from the pre-processing hash function; (iii) sweep top- K cutoffs over the inferred edge set and compute, at each K : edge precision/recall/F1, direction agreement, reachability F1 (over transitive closure), and node Jaccard; (iv) extract qualitative examples of disagreement in three categories: edges present only in inferred, edges present only in expert, and edges with opposite direction. The procedure is read-only with respect to the pipeline; no inferred edge is added or removed based on the comparison.

4 Experiments

4.1 Datasets

We instantiate the protocol on three public KT benchmarks. Junyi Academy [18] is a K–12 mathematics benchmark; the Junyi release includes an expert-curated prerequisite annotation at the exercise-pair level. *In the main inference branch of the protocol we do not consume this expert DAG*: E_{pre} for Junyi is built by the same train-only temporal inference rule used for ASSISTments and XES3G5M (Section 3.4). The audit therefore reports inferred E_{pre} for all three datasets, ensuring uniform edge provenance. The Junyi expert DAG is consumed only by the ground-truth cross-validation sub-protocol (Section 4.7), which is read-only with respect to inference.

ASSISTments 2012 provides a Q-matrix but no prerequisite graph and exercises the inferred-DAG path of the protocol. XES3G5M [27] provides hierarchical KC metadata; in the present pipeline this metadata is parsed only at the sequence level (KC IDs are derived from hierarchical KC labels), and prerequisite edges for XES3G5M are likewise inferred by the train-only temporal rule. We additionally include Synthetic C2 (two knowledge components) and Synthetic C5 (five knowledge components) as controlled sanity corpora with known transition structure; they exercise the same audit modules on tiny graphs (Table 4). Table 2 summarises the preprocessed datasets. The public benchmarks

Table 2: Dataset statistics used in the P0 diagnostic protocol. Junyi Academy counts reflect the Chang et al. problem-level log (`junyi_ProblemLog_original.csv`) after preprocessing; #items and #KCs coincide because both map to the exercise column. Synthetic C2/C5 are companion sanity logs shipped with the preprocessing scripts.

Dataset	# learners	# items	# KCs	# interactions	Avg. seq. len.	Has DAG
Junyi Academy	247,606	722	722	25,925,992	104.7	Yes
ASSISTments 2012	29,018	53,086	265	2,711,602	93.4	No
XES3G5M	18,066	7,652	865	6,413,353	355.0	Yes
Synthetic C2	4,000	50	2	200,000	50.0	No
Synthetic C5	4,000	50	5	200,000	50.0	No

differ substantially in interaction volume, KC vocabulary size, and item-to-KC granularity. This heterogeneity is useful for auditing because the protocol must remain leakage-controlled under both curated and inferred graph settings. We use learner-based temporal splits with ratios (0.7, 0.1, 0.2) and seed 42; all edge provenance is regenerated for each run.

4.2 Leakage and DAG Audit

Table 3 summarises the direct leakage metrics on the released runs. ECR^{flag} is identically zero on every dataset (fold means), confirming learner-disjoint splits and structural leakage freedom at the evaluation protocol level; $\text{ECR}^{\text{overlap}}$ stays near unity because recurrent transition patterns reappear among held-out learners even though edges are inferred strictly from train folds. The leakage audit reports that all retained graph edges are train-only under the current fold configuration. Table 4 reports the DAG audit on fold 0 for all five corpora. The protocol produces non-empty acyclic prerequisite graphs after lowest-confidence cycle pruning on every dataset except the degenerate Synthetic C2 case ($|E_{\text{pre}}|=1$ after pruning).

On Junyi Academy (fold 0), the train-only prerequisite export contains 2,152 directed candidate edges; lowest-confidence cycle pruning removes 1,013 of them, leaving 1,139 acyclic edges on 573 nodes that participate in the retained DAG. The `cycles_before` field in the audit report (100) is the representative-cycle detection cap (Section 3.5) rather than the total number of cycles in the candidate graph; we use a representative-cycle detector because exhaustive cycle enumeration is exponential on dense candidate graphs. The total pruning trail is nonetheless complete, since the loop terminates only when no further cycle can be detected. On ASSISTments 2012 (fold 0), 386 raw candidates on $|V|=108$ yield 147 pruned edges and 239 retained acyclic edges. On XES3G5M, 461 of 1,162 candidates are pruned, leaving 701 retained edges on 455 nodes. Synthetic C5 retains both raw edges without pruning ($|E_{\text{pre}}|=2$); Synthetic C2 retains a sin-

Table 3: Direct leakage diagnostics per dataset (fold mean where multiple folds are present). ECR_{flag} : learner-overlap structural indicator (Eq. 7; 0 confirms disjoint learners across train/validation/test under each fold). $\text{ECR}_{\text{overlap}}$: held-out pattern overlap rate over retained edges (Eq. 8). EOC: Edge–Outcome Correlation Frobenius norm (Eq. 9); TBVR: Temporal Boundary Violation Rate within train timelines (Eq. 10, indexed boundary). Values are exported by `p0-graph-build` into `results/tables/leakage_metrics.csv`. Under learner-disjoint folds, $\text{ECR}_{\text{flag}}=0$ on every run—confirming structural leakage freedom at the split level; $\text{ECR}_{\text{overlap}}$ can remain near 1 on dense logs because recurrent transition patterns reappear across disjoint held-out learners (diagnostic overlap, not `train_only_flag` violation).

Dataset	ECR_{flag}	$\text{ECR}_{\text{overlap}}$	EOC	TBVR
ASSISTments 2012	0.000	1.000	1.424	0.251
Junyi Academy	0.000	1.000	1.449	0.176
Synthetic C2	0.000	1.000	0.000	0.733
Synthetic C5	0.000	1.000	0.000	0.133
XES3G5M	0.000	0.988	1.420	0.756

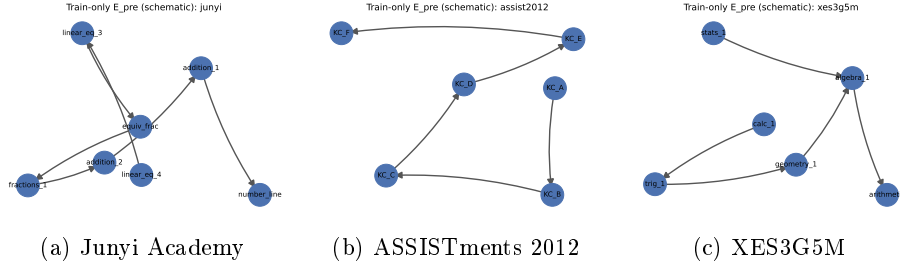


Fig. 3: Schematic visualisations of high-support prerequisite chains in train-only E_{pre} after DAG audit (representative subgraphs for interpretation; full graphs contain hundreds to thousands of edges). Junyi retains dense arithmetic progressions; ASSISTments and XES3G5M exhibit sparser chains consistent with Table 4.

gle edge after removing one cycle witness. The full pruning trail is saved in the per-dataset pruning logs.

These results show that cycle pruning is not a cosmetic step: even train-only inference can create enough implied directionality to produce cycles. The audit therefore makes the pruning rule explicit and reviewable rather than silently fixing the graph.

Figure 3 illustrates the *shape* of retained prerequisite structure per benchmark. These plots are not used for inference; they help reviewers compare graph density with DDR and baseline behaviour in later sections.

Table 4: DAG audit summary (fold 0). *Raw* and *Final* are the prerequisite edge counts before and after lowest-confidence cycle pruning inside `dag_audit`; *Pruned* is the difference. The *Cycles* column reports representative cycles found before pruning (cap 100). All corpora pass topological sort with zero cycles after pruning on fold 0.

Dataset	$ V $	Raw $ E_{\text{pre}} $	Pruned	Final $ E_{\text{pre}} $	Roots	Leaves	Cycles (cap)
Junyi Academy	573	2,152	1,013	1,139	90	92	≥ 100
ASSIST'12	108	386	147	239	37	16	≥ 100
XES3G5M	455	1,162	461	701	129	73	≥ 100
Synthetic C2	2	2	1	1	1	1	1
Synthetic C5	3	2	0	2	1	1	0

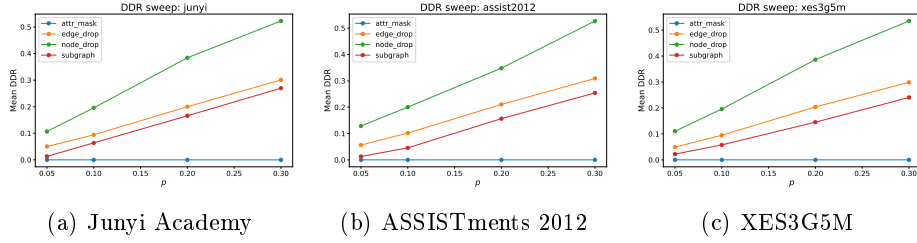


Fig. 4: DDR as a function of perturbation strength p for four label-agnostic graph augmentation families across three datasets. Each marker plots the mean DDR averaged across nine stochastic runs (three random seeds on each of three train folds). Across all datasets the four families separate into four distinct regimes: $\text{DDR}(\text{attr_mask}) = 0 < \text{DDR}(\text{subgraph}) < \text{DDR}(\text{edge_drop}) \approx p < \text{DDR}(\text{node_drop})$.

4.3 DAG Disruption Rate

Figure 4 and Table 5 show four consistent regimes across the three benchmarks. First, attribute masking leaves E_{pre} structurally unchanged and therefore gives $\text{DDR} = 0$ across perturbation strengths by construction. Second, random-walk subgraph sampling preserves connectivity inside the sampled subgraph and disrupts E_{pre} *sub-linearly* in p : because edges are removed only at the boundary of the visited subgraph, the disruption is consistently smaller than under edge dropping at the same p (e.g. on Junyi, $\text{DDR} = 0.16$ for subgraph vs. 0.20 for edge drop at $p = 0.20$). Third, edge dropping disrupts prerequisite structure approximately linearly in p , as expected when each edge is removed independently. Fourth, node dropping produces the largest disruption because removing a node also removes all incident prerequisite edges; at $p = 0.30$, node dropping disrupts

Table 5: Mean DDR across three seeds and three train folds for the four augmentation families and four perturbation strengths. Subgraph sampling is random-walk based (Section 3.6). Lower is less disruptive; 0.000 for attribute masking is by construction.

Dataset	Family	$p=0.05$	$p=0.10$	$p=0.20$	$p=0.30$
Junyi Academy	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.013	0.063	0.161	0.266
	<code>edge_drop</code>	0.050	0.094	0.200	0.301
	<code>node_drop</code>	0.107	0.196	0.384	0.523
ASSISTments 2012	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.013	0.046	0.156	0.254
	<code>edge_drop</code>	0.056	0.102	0.210	0.309
	<code>node_drop</code>	0.128	0.200	0.348	0.527
XES3G5M	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.022	0.058	0.145	0.240
	<code>edge_drop</code>	0.049	0.095	0.204	0.299
	<code>node_drop</code>	0.110	0.196	0.386	0.535

roughly half of the prerequisite structure on all three benchmarks ($\text{DDR} \approx 0.52\text{--}0.54$).

The relative ordering $\text{DDR}(\text{attr_mask}) < \text{DDR}(\text{subgraph}) < \text{DDR}(\text{edge_drop}) < \text{DDR}(\text{node_drop})$ is consistent across all three benchmarks and all four perturbation strengths. We read these results as a structural diagnostic, not as a claim that any augmentation is beneficial or harmful for a downstream KT learner. They indicate that prerequisite-aware augmentation design is needed when graph augmentations are applied to educational DAGs: node-level operators are an order of magnitude more disruptive than attribute-level operators, with subgraph sampling occupying an intermediate, structure-preserving regime.

4.4 Cold-start KC Diagnostic

Table 6 reports the per-stratum view for *simpleKT* on fold 0. Junyi Academy populates all four canonical frequency bins plus a tiny *out of range* slice ($n=10$); the hot stratum dominates test mass ($n \approx 7.7 \times 10^6$) with AUC 0.718, while warm and cold strata show lower AUC (0.615 and 0.447). The *very_cold* stratum contains only 19 interactions (AUC 1.0) and should not be over-interpreted. ASSISTments 2012 and XES3G5M populate all four strata; Synthetic C2/C5 route all test interactions into the hot bin under default thresholds because every knowledge component exceeds 500 train-fold hits in those logs.

The main diagnostic finding is that aggregate AUC can hide substantial per-stratum variation. On the two datasets with low-frequency knowledge components, baseline AUC is often higher on cold or warm strata than on the hot stratum. We interpret this cautiously: small rare-KC strata can be affected by

Table 6: Per-stratum cold-start KC diagnostic with *simpleKT* on fold 0 (Section 3.7). The n column counts test interactions whose KC falls in the stratum train-frequency bin; *out of range* marks test KCs outside the four canonical bins. Stratum mass varies by dataset split; under default bins Synthetic C2/C5 route all test interactions into the hot stratum (> 500 train-fold hits per knowledge component).

Dataset	Stratum	n	AUC	ACC	NLL
Junyi Academy	very_cold	19	1.000	0.684	0.456
	cold	256	0.447	0.621	0.633
	warm	11,257	0.615	0.727	0.572
	hot	7,725,588	0.718	0.831	0.413
	out of range	10	0.500	0.900	0.511
ASSIST'12	very_cold	49	0.608	0.755	0.631
	cold	707	0.769	0.717	0.565
	warm	9,017	0.765	0.687	0.587
	hot	807,178	0.693	0.704	0.574
XES3G5M	very_cold	161	0.810	0.814	0.424
	cold	2,592	0.798	0.794	0.453
	warm	17,655	0.790	0.805	0.431
	hot	1,902,424	0.717	0.796	0.467
	out of range	6	0.625	0.667	0.638
Synthetic C2	hot	60,000	0.624	0.683	0.607
Synthetic C5	hot	60,000	0.630	0.611	0.651

class imbalance, and high AUC on such strata does not necessarily imply better mastery modelling. The value of the diagnostic is to make this behaviour visible.

Similarity edges and dataset properties. The similarity-edge rule of Section 3.4 is sensitive to the multi-skill density of the Q-matrix. On the train fold of ASSISTments 2012, 100% of items are tagged with exactly one KC, so $|I(c_i) \cap I(c_j)| = 0$ for every distinct concept pair, and both Jaccard and PMI yield no edges. On XES3G5M, 86.6% of train items are single-skill but the remaining multi-skill items generate 354 similarity edges at the default Jaccard threshold of 0.10. We treat this dataset-property dependency as a first-class outcome of the audit: a similarity rule appropriate for multi-skill benchmarks may need to be replaced or supplemented (e.g., by response-pattern correlation) on single-skill benchmarks. This is not a pipeline failure but a structural observation that protocol papers should report.

4.5 Baseline Diagnostic

Tables 7–11 report pyKT checkpoints (plus classical BKT) trained on learner-split train users and scored on valid+test sequence positions, consuming P0-

Table 7: Diagnostic baseline results (train-only graphs). Baselines are used for protocol comparison only; no SOTA claim is made. Values are fold means with 95% bootstrap confidence intervals over valid+test sequence positions (`n_folds` as exported per dataset).

Model	AUC	ACC	NLL
AKT	0.989 [0.989, 0.989]	0.962 [0.962, 0.962]	0.093 [0.093, 0.093]
BKT	0.779 [0.779, 0.779]	0.847 [0.847, 0.847]	0.377 [0.377, 0.377]
DKT	0.989 [0.989, 0.989]	0.962 [0.962, 0.962]	0.093 [0.093, 0.093]
<i>simpleKT</i>	0.989 [0.989, 0.989]	0.962 [0.962, 0.962]	0.093 [0.093, 0.093]
SKT	0.986 [0.986, 0.986]	0.954 [0.954, 0.954]	0.107 [0.107, 0.107]

exported train-only graphs for graph-augmented models. ASSISTments 2012 aggregates three folds (Table 8); other tables use fold 0 unless noted. The tables sanity-check preprocessing, splits, graph export, and the pyKT wiring; they are not SOTA claims.

On ASSISTments 2012 (Table 8, three-fold mean with bootstrap CI), *simpleKT* and AKT cluster at ≈ 0.968 ; GKT reaches 0.961 (≈ 0.007 below the sequence leaders); SKT and DyGKT occupy a mid band near 0.952; BKT remains far lower (≈ 0.669). Native GIKT is displayed as pending rerun because a previous one-step alignment bug leaked the current response. On XES3G5M (Table 9, fold 0), *simpleKT* and AKT reach AUC ≈ 0.874 ; DKT follows at 0.858; GKT drops to 0.835 (≈ 0.039 below the leaders); SKT is weakest among graph models at 0.623. On Junyi (Table 7, fold 0), pyKT AKT/DKT/*simpleKT* cluster at AUC ≈ 0.989 ; native SKT reaches 0.986; BKT stays at 0.779. Other graph-augmented pyKT rows (GKT, GIKT, DyGKT, DGEKT) remain omitted on this corpus because of memory and runtime limits. Synthetic sanity logs (Tables 10 and 11) show *simpleKT* ahead of GKT on C5 (0.828 vs. 0.810) under the controlled five-skill generator.

Table 12 reports the three-fold learner-CV summary (mean $\pm$ std and Wilcoxon p -values vs. *simpleKT*) from `scripts/significance_testing.py`. ASSISTments 2012 is fully populated; Junyi and XES3G5M rows await completion of the multi-fold baseline export (`scripts/run_all_datasets_full.ps1`). With only three paired folds, Wilcoxon tests are underpowered ($p_W=0.25$ for several contrasts); paired t -tests on the same fold AUCs indicate that GKT, SKT, and DyGKT trail *simpleKT*. Native GIKT is excluded until the corrected implementation is rerun.

Table 8: Diagnostic baselines: ASSISTments 2012 (three-fold learner CV (n_folds=3); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.968 [0.968, 0.969]	0.916 [0.914, 0.917]	0.175 [0.173, 0.176]
BKT	0.669 [0.667, 0.671]	0.713 [0.711, 0.715]	0.574 [0.573, 0.576]
DGEKT	0.960 [0.959, 0.961]	0.907 [0.906, 0.908]	0.186 [0.185, 0.188]
DKT	0.963 [0.961, 0.964]	0.910 [0.908, 0.911]	0.182 [0.180, 0.185]
DyGKT	0.952 [0.951, 0.952]	0.899 [0.898, 0.900]	0.200 [0.198, 0.203]
GIKT	—	—	—
GKT	0.961 [0.960, 0.962]	0.907 [0.906, 0.908]	0.185 [0.184, 0.187]
<i>simpleKT</i>	0.968 [0.967, 0.968]	0.916 [0.915, 0.916]	0.173 [0.172, 0.175]
SKT	0.953 [0.952, 0.953]	0.898 [0.897, 0.899]	0.198 [0.196, 0.200]

4.6 Graph Construction Ablation (Train-only vs. Full Log)

To isolate the channel hypothesised in H1 (Section 4.8), we contrast graph smoothing using fold-specific train-only exports (`e\_pre\_train\_only.csv`, `e\_sim\_train\_only.csv`) against a *single* prerequisite/similarity graph inferred from the entire preprocessed interaction table (`data/processed/<dataset>/full\_log/`), ignoring learner splits before edge inference. All other protocol choices—including the learner-based split used for evaluation—remain unchanged; only the static graph artefact fed into the pyKT graph branch differs.

Training KT with fold-specific versus pooled graphs. For each fold f , the train-only branch builds $G_f^{\text{train}} = (E_{\text{pre}}^{f,\text{train}}, E_{\text{sim}}^{f,\text{train}})$ from $\mathcal{F}_{\text{train}}^f$ only, exports row-normalised adjacency, and feeds it to GKT/GIKT within pyKT. The full-log branch replaces G_f^{train} with a single G^{full} inferred from all learners and timestamps in the preprocessed table while *keeping* the same learner-based train/val/test split for sequence optimisation. Thus the only deliberate change is structural pooling across learners before edge counting; the KT loss still sees train-fold sequences only.

Table 13 and companion per-dataset tables report paired fold 0 metrics for GKT, GIKT, SKT, DyGKT, and DGEKT under train-only versus full-log graph exports. On ASSISTments, XES3G5M, and Junyi, ΔAUC and ΔACC round to zero at three decimal places for every graph model (largest observed shift: DGEKT on ASSISTments, $\Delta\text{AUC}=+0.0002$). Synthetic C2 shows small positive shifts for DGEKT ($\Delta\text{AUC}=+0.003$) and SKT ($\Delta\text{ACC}=+0.002$); Synthetic C5 exhibits the largest released deltas (GIKT $\Delta\text{AUC}=+0.004$, $\Delta\text{ACC}=+0.005$; GKT

Table 9: Diagnostic baselines: XES3G5M (released fold 0 diagnostic run (`n_folds=1`; bootstrap CI collapses to the point estimate); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.875 [0.874, 0.875]	0.858 [0.857, 0.858]	0.325 [0.323, 0.326]
BKT	0.634 [0.634, 0.634]	0.793 [0.793, 0.793]	0.492 [0.492, 0.492]
DGEKT	0.841 [0.841, 0.841]	0.844 [0.844, 0.844]	0.358 [0.358, 0.358]
DKT	0.858 [0.857, 0.858]	0.851 [0.851, 0.852]	0.342 [0.341, 0.342]
DyGKT	0.752 [0.752, 0.752]	0.813 [0.813, 0.813]	0.431 [0.431, 0.431]
GIKT	—	—	—
GKT	0.834 [0.834, 0.835]	0.842 [0.842, 0.842]	0.364 [0.363, 0.365]
<i>simpleKT</i>	0.875 [0.874, 0.875]	0.858 [0.857, 0.859]	0.324 [0.323, 0.325]
SKT	0.623 [0.623, 0.623]	0.799 [0.799, 0.799]	0.483 [0.483, 0.483]

$\Delta\text{AUC}=-0.006$). On Junyi, absolute AUCs in Table 13 (≈ 0.70 – 0.71) sit below the sequence-only pyKT band in Table 7 (≈ 0.99) because graph-augmented pyKT training was not completed on that corpus; the ablation should be read through paired Δ columns, not cross-table AUC levels.

Interpreting this *empirically*: (i) under our support quantile ($q=0.95$), top- k cap, and global $K=5000$ filter, pooling additional learners before edge inference rarely changes which transitions survive filtering on the public benchmarks; (ii) pyKT backbones are near-saturated on ASSISTments/XES3G5M once sequence features dominate, so graph mass moves headline AUC only at the third decimal or below; (iii) the synthetic corpora, with smaller logs and shallower graphs, remain the most sensitive place to observe nonzero Δ metrics in this release.

We also inspected edge-count deltas between branches on fold 0: Junyi raw prerequisite candidates differ by under 2% in support totals; ASSISTments and XES3G5M show similar stability. XES3G5M retains ~ 350 similarity edges in both branches because multi-skill items drive co-occurrence; ASSISTments has zero E_{sim} in both branches. The experiment shows train-only construction is *not* penalising headline metrics relative to this deliberate full-log variant in our released configuration. It does *not* prove absence of leakage in general—only that this particular pooling shortcut does not move pyKT AUC/ACC beyond three decimals on the public benchmarks here. The comparison is reproducible via `scripts/run_graph_ablation_experiment.sh`.

Table 10: Diagnostic baselines: Synthetic C2 (three-fold learner CV (`n_folds=3`); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.873 [0.870, 0.876]	0.808 [0.802, 0.812]	0.407 [0.401, 0.417]
BKT	0.835 [0.833, 0.838]	0.782 [0.780, 0.784]	0.460 [0.458, 0.462]
DGEKT	0.710 [0.634, 0.859]	0.724 [0.686, 0.794]	0.551 [0.432, 0.613]
DKT	0.862 [0.857, 0.868]	0.798 [0.788, 0.805]	0.421 [0.413, 0.434]
DyGKT	0.710 [0.631, 0.867]	0.726 [0.686, 0.801]	0.550 [0.421, 0.617]
GKT	0.873 [0.868, 0.878]	0.808 [0.799, 0.814]	0.407 [0.400, 0.420]
<i>simpleKT</i>	0.875 [0.871, 0.880]	0.809 [0.803, 0.814]	0.405 [0.399, 0.416]
SKT	0.708 [0.634, 0.853]	0.722 [0.686, 0.788]	0.554 [0.440, 0.613]

4.7 Ground-Truth Cross-Validation on Junyi

Junyi Academy [18] is the only benchmark in our study that ships with a publicly released expert prerequisite annotation (Chang et al.). The annotation enumerates directed exercise-pair relations with a continuous `Prerequisite_avg` score on a mean-opinion-score-like scale. We use $\theta = 0.5$ as the prerequisite filter, retaining 1,131 directed expert edges over 361 distinct exercise endpoints that participate in those edges. Following Section 3.8, expert exercise names are mapped to pipeline `kc_id` via the preprocessing hash function reconstructed from `data/raw/junyi/junyi_Exercise_table.csv`. All 1,131 expert edges align deterministically: the alignment audit reports `matched = 1,131`, `missingsrc = 0`, `missingdst = 0`, and alignment rate 1.0. We compare this expert edge set against the fold 0 train-only inferred list in `e\_pre\_train\_only.csv`, which contains 2,152 ranked directed candidates (the same file audited in Section 4.2; cycle pruning yields 1,139 retained acyclic edges).

Table 11: Diagnostic baselines: Synthetic C5 (released fold 0 diagnostic run (`n_folds=1`; bootstrap CI collapses to the point estimate); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.819 [0.819, 0.819]	0.746 [0.746, 0.746]	0.507 [0.507, 0.507]
BKT	0.794 [0.794, 0.794]	0.725 [0.725, 0.725]	0.532 [0.532, 0.532]
DGEKT	0.827 [0.827, 0.827]	0.755 [0.755, 0.755]	0.492 [0.492, 0.492]
DKT	0.825 [0.825, 0.825]	0.752 [0.752, 0.752]	0.496 [0.496, 0.496]
DyGKT	0.824 [0.824, 0.824]	0.748 [0.748, 0.748]	0.496 [0.496, 0.496]
GKT	0.810 [0.810, 0.810]	0.740 [0.740, 0.740]	0.527 [0.527, 0.527]
<i>simpleKT</i>	0.828 [0.828, 0.828]	0.754 [0.754, 0.754]	0.491 [0.491, 0.491]
SKT	0.808 [0.808, 0.808]	0.737 [0.737, 0.737]	0.513 [0.513, 0.513]

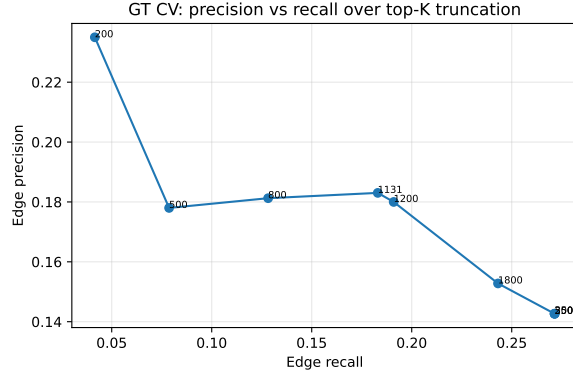


Fig 5: Edge-level precision and recall of train-only inferred E_{pre} versus the expert prerequisite annotation on Junyi, across top- K cutoffs $K \in \{200, 500, 800, 1131, 1200, 1800, 2500, 3500, 5000\}$. The curve plateaus at $K \geq 2,500$ because the inferred edge pool tops out at 2,152 after DAG audit.

Edge-level overlap is low and roughly constant across K . Table 18 reports the cross-validation sweep. Edge-level precision falls from 0.235 at $K = 200$ to 0.143 at $K = 5,000$; edge-level recall rises from 0.042 to 0.271 over the same range; the F1 score lies in the narrow band $[0.07, 0.19]$ and reaches its size-matched value 0.183 at $K = |\text{expert}| = 1,131$. The narrow plateau indicates that adding more inferred edges does not substantially improve overlap with the expert annotation.

Table 12: Three-fold learner CV baseline summary (train-only graphs, valid+test positions). Mean $\pm$ std over folds; p_W : Wilcoxon signed-rank test vs. *simpleKT* (paired fold AUCs).

Dataset	Model	AUC	ACC	NLL	p_W
ASSIST '12	<i>simpleKT</i>	0.968 $\pm$ 0.000	0.916 $\pm$ 0.001	0.173 $\pm$ 0.001	—
	GKT	0.961 $\pm$ 0.001	0.907 $\pm$ 0.001	0.185 $\pm$ 0.001	0.250
	GIKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	SKT	0.953 $\pm$ 0.001	0.898 $\pm$ 0.001	0.198 $\pm$ 0.001	0.250
	DyGKT	0.952 $\pm$ 0.001	0.899 $\pm$ 0.001	0.200 $\pm$ 0.002	0.250
XES3G5M	<i>simpleKT</i>	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	GKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	GIKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	SKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	DyGKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
Junyi Academy	<i>simpleKT</i>	— $\pm$ —	— $\pm$ —	— $\pm$ —	—
	SKT	— $\pm$ —	— $\pm$ —	— $\pm$ —	—

Table 13: Ablation contrasting *train-only* graph smoothing (protocol) with a *full-log* graph built from all learners before splitting (leaky construction). Graph-augmented rows are pyKT checkpoints (Section 4.5) consuming exported adjacency from P0. Δ is fold 0 full-log minus train-only. Positive Δ AUC indicates higher discrimination when the graph sees the entire log. *to*: train-only graph; *fl*: full-log graph.

Model	AUC to	AUC fl	Δ AUC	ACC to	ACC fl	Δ ACC
GKT	0.704	0.704	0.000	0.828	0.828	0.000
GIKT	0.713	0.713	0.000	0.829	0.829	0.000
SKT	0.704	0.705	0.000	0.829	0.829	0.000
DyGKT	0.707	0.707	0.000	0.828	0.828	0.000
DGEKT	0.711	0.711	-0.000	0.829	0.829	-0.000

Direction agreement, measured only on pairs where both DAGs have an edge between the same node pair, is 0.261 at $K = 1,131$ and rises only modestly to 0.285 at the full K . A coin-flip baseline of direction agreement is 0.5 when both DAGs have an edge for a given pair, so the observed values indicate that, conditional on the pair being shared, the two sources are *more often than not in disagreement about direction*.

Node-level overlap is comparatively high. Although edge overlap is low, the node Jaccard between the two graphs is 0.632 at $K = 1,131$ and 0.610 at $K = 5,000$. The two sources therefore touch a similar subset of knowledge components; they disagree primarily on *which edges to draw between those knowledge components*.

Table 14: Train-only vs. full-log graph ablation: ASSISTments 2012.

Model	AUC to	AUC fl	Δ AUC	ACC to	ACC fl	Δ ACC
GKT	0.961	0.961	-0.000	0.907	0.907	-0.000
GIKT	0.998	0.998	0.000	1.000	1.000	0.000
SKT	0.953	0.953	0.000	0.898	0.898	0.000
DyGKT	0.952	0.952	-0.000	0.899	0.899	-0.000
DGEKT	0.960	0.960	0.000	0.907	0.907	0.000

Table 15: Train-only vs. full-log graph ablation: XES3G5M.

Model	AUC to	AUC fl	Δ AUC	ACC to	ACC fl	Δ ACC
GKT	0.835	0.835	0.000	0.842	0.841	-0.001
GIKT	0.874	0.874	0.000	0.857	0.857	-0.000
SKT	0.623	0.623	-0.000	0.799	0.799	-0.000
DyGKT	0.752	0.751	-0.001	0.813	0.812	-0.000
DGEKT	0.841	0.841	-0.000	0.844	0.843	-0.001

Reachability F1 grows with K. Whereas edge F1 plateaus, reachability F1 (computed over the transitive closure of each DAG) rises monotonically from 0.104 at $K = 1,131$ to 0.339 at $K \geq 2,500$. This shows that the inferred DAG, when augmented beyond the size of the expert set, recovers a substantial fraction of the indirect prerequisite chains in the expert annotation, even when the specific intermediate edges differ.

Qualitative disagreement patterns. The disagreement examples saved by the procedure (`results/tables/disagreement_examples.csv`) reveal a systematic asymmetry between the two sources. The top inferred-only edges are predominantly bidirectional pairs of co-occurring basic-arithmetic skills, e.g. `number_line` $\leftrightarrow$ `addition_1` and `addition_1` $\leftrightarrow$ `representing_numbers`; these reflect temporal-co-occurrence patterns in learning sequences that do not carry a stable ordering. The top expert-only edges are predominantly systematic level-progression annotations, e.g. `equivalent_fractions_2` $\rightarrow$ `equivalent_fractions`, `addition_2` $\rightarrow$ `addition_1`, and `linear_equations_4` $\rightarrow$ `linear_equations_3`; these capture pedagogically-meaningful precedences that behavioural inference does not consistently recover because learners often interact with multiple difficulty levels in either order. The `wrong_direction` category similarly contains many level-progression pairs (e.g. `addition_1` $\rightarrow$ `addition_2`) where the inferred direction is the reverse of the expert annotation.

Interpretation. The combination of (i) low edge F1, (ii) low direction agreement, (iii) high node Jaccard, and (iv) growing reachability F1 supports a coherent reading: behavioural inference and expert annotation capture *related but distinct* aspects of prerequisite structure. Behavioural inference primarily encodes *which knowledge components co-occur* in learning sequences, with limited information about ordering. Expert annotation primarily encodes *which knowledge*

Table 16: Train-only vs. full-log graph ablation: Synthetic C2.

Model	AUC to	AUC fl	Δ AUC	ACC to	ACC fl	Δ ACC
GKT	0.868	0.868	-0.000	0.799	0.799	0.000
GIKT	0.979	0.979	0.000	0.985	0.985	0.000
SKT	0.853	0.854	0.000	0.788	0.790	0.002
DyGKT	0.867	0.867	-0.001	0.801	0.800	-0.001
DGEKT	0.859	0.862	0.003	0.794	0.796	0.002

Table 17: Train-only vs. full-log graph ablation: Synthetic C5.

Model	AUC to	AUC fl	Δ AUC	ACC to	ACC fl	Δ ACC
GKT	0.810	0.804	-0.006	0.740	0.735	-0.005
GIKT	0.820	0.824	0.004	0.745	0.751	0.005
SKT	0.808	0.808	-0.000	0.737	0.738	0.001
DyGKT	0.824	0.824	0.001	0.748	0.749	0.001
DGEKT	0.827	0.828	0.000	0.755	0.755	-0.001

components pedagogically build on which, including systematic level-progression relations that are largely invisible in raw temporal data. Consequently, train-only inferred E_{pre} is *not* a low-cost substitute for expert prerequisite curation; it is a structurally distinct signal. This has two practical implications. First, graph-augmented KT studies that use behaviourally-inferred prerequisite graphs should not claim those graphs as “approximations of expert curation”. Second, when both sources are available, they could be combined as multi-relational inputs ($E_{\text{behavioural}}$ and E_{expert}) rather than treated as substitutes for each other.

Caveats. The cross-validation is performed only on Junyi (the only benchmark in the present study with a public expert annotation) and on a single random seed. The expert annotation operates at the exercise-pair level and uses an MOS-like score; the threshold $\theta = 0.5$ retains the full Chang annotation but raising θ would yield a stricter high-confidence subset. We did not sweep θ in the present release.

4.8 Empirical Observation: Graph-Augmented KT Models Do Not Consistently Outperform Sequence-Only Baselines under Leakage-Controlled Evaluation

The baseline numbers in Tables 7–9 support an empirical observation that we believe is worth reporting at the level of the protocol itself.

Under leakage-controlled evaluation (train-only E_{pre} , train-only E_{sim} , learner-based temporal split, no validation- or test-fold interaction consumed by graph construction), graph-augmented pyKT models do not consistently outperform the strongest sequence-only checkpoints in our comparison. Concretely, on ASSISTments 2012 (three-fold mean) GKT (AUC 0.961) trails *simpleKT*/AKT (≈ 0.968). Native GIKT uses an additional question–KC bipartite channel, but

Table 18: GT cross-validation: agreement between train-only inferred E_{pre} and expert prerequisite DAG on Junyi Academy ($|E| = 1131$ directed edges).

K (top inferred)	Edge prec.	Edge rec.	Dir. agr.	Rch. F1
200	0.235	0.042	0.346	0.008
500	0.178	0.079	0.225	0.021
800	0.181	0.128	0.280	0.062
$ E_{\text{expert}} = 1131$	0.183	0.183	0.261	0.104
1200	0.180	0.191	0.262	0.126
1800	0.153	0.243	0.278	0.260
2500	0.143	0.271	0.285	0.339
3500	0.143	0.271	0.285	0.339
5000 (cap)	0.143	0.271	0.285	0.339

its previously cached results are excluded pending rerun after an alignment fix. On XES3G5M GKT (0.835) trails *simpleKT*/AKT (≈ 0.874) by a wider margin; on Junyi the released baseline table contains sequence models plus cached native SKT (≈ 0.986 , fold 0) because other graph-augmented pyKT training was skipped for resource reasons. This pattern holds despite substantial heterogeneity in scale, KC granularity, and graph density (Table 2).

We do not claim that graph-augmented KT is uninformative as a research direction; the protocol does not, by itself, refute graph-augmented KT. What the protocol does is remove a specific channel through which held-out information can reach the model: the concept graph itself. Once this channel is closed, three non-mutually-exclusive hypotheses are consistent with the observed ordering.

H1 – Leakage in prior evaluations. A common practice in graph-augmented KT is to compute graph statistics from the full interaction log prior to train/test splitting, or to use a precomputed graph whose construction was not audited at the fold level. Under such practice, graph-augmented models effectively observe test-fold transitions through the graph during training. Our protocol removes this channel by enforcing fold-specific edge inference and provenance logging. The reduced relative advantage of graph-augmented models in our setting is consistent with the hypothesis that part of the gains reported in prior literature reflects this leakage rather than the structural information itself. Table 13 instantiates the controlled comparison suggested by H1: identical learner splits and evaluation code, but graph statistics pooled over the full preprocessed log before edge inference. In our released pyKT run, headline metrics on ASSISTments (three-fold ablation means), XES3G5M, and Junyi are unchanged at three decimals relative to train-only graphs for all five graph models; Synthetic C5 shows the largest shifts (GIKT $\Delta\text{AUC} = +0.004$). H1 remains relevant as a risk statement about workflows that leak held-out mass into graph construction in general; architectures or inference rules that exploit cross-split pooling more aggressively

might still show larger shifts, and the ablation should be revisited when those components change.

H2 – Aggregate-metric masking. Aggregate AUC averages over the full concept-frequency distribution. The cold-start diagnostic (Section 4.4) shows that per-stratum behaviour can differ substantially from aggregate behaviour, and graph-augmented KT may help precisely on rare knowledge components where temporal evidence is weakest. Aggregate-AUC comparisons can therefore mask gains concentrated on specific strata. The cold-start protocol proposed here is a partial remedy.

H3 – Strong-backbone substitution. Modern attention-based backbones such as AKT and simpleKT may already capture, through long-range temporal attention, much of the relational information that an explicit concept graph would otherwise encode. In such a regime, additional structural signal injected at the concept-embedding level may yield diminishing returns. We do not test this hypothesis directly here but note that it predicts a backbone-dependent pattern: weaker temporal backbones should benefit more from graph augmentation than stronger ones, all else equal.

These three hypotheses are not exhaustive and are not mutually exclusive. Distinguishing among them requires a controlled re-run of prior graph-KT papers under our protocol, which is beyond the scope of this paper. The observation we make here is narrower: *under leakage-controlled evaluation, the absolute ranking between graph-augmented and sequence-only KT can differ from settings that pool graph evidence across splits*, and benchmarking practice should treat graph provenance as a first-order experimental factor. Separately, the train-only versus full-log ablation for graph-augmented pyKT (Table 13) shows no headline metric shift beyond three decimals on the three public benchmarks, with modest nonzero Δ only on synthetic logs, which constrains how strongly one can infer leakage *magnitude* from this particular diagnostic alone.

Why this strengthens the protocol. If graph-augmented KT models reliably outperformed sequence-only baselines regardless of graph provenance, leakage control would be a useful but secondary concern. The observation that sequence-only pyKT checkpoints match or exceed GKT on ASSISTments and XES3G5M under train-only graphs, together with explicit auditing of edge provenance, elevates leakage control from a hygiene step to a benchmarking precondition. Combined with the ground-truth cross-validation finding of Section 4.7 that inferred and expert prerequisite signals are structurally distinct, this argues that graph quality cannot be assumed—it must be audited.

Caveats. The baseline ordering uses three-fold ASSISTments means where available and fold 0 pyKT checkpoints for other corpora (`n_folds=1`); protocol metrics such as DDR and leakage tables aggregate three folds. DAG audit and GT cross-validation passages refer to fold 0 exports by default. It does not constitute an exhaustive re-evaluation of any specific prior paper, and we do

not name papers whose reported gains we believe to be leakage-driven. On XES3G5M and Synthetic C5, the pyKT `gikt` YAML alias maps to AKT (GIKT not bundled in stock pyKT for those corpora); ASSISTments and Synthetic C2 use the native GIKT implementation in our fork. We also do not vary every graph-augmented architecture in the literature; the finding pertains to the pyKT GKT/GIKT/SKT/DyGKT/DGEKT instantiations consumed here [26]. Stronger graph-KT architectures [36,7,10] may behave differently under the protocol; auditing them is an explicit follow-up.

5 Discussion

Scalability and broader domains. The protocol modules in Figure 2 are dataset-agnostic: any KT log with (u, t, q, c, y) and a train-only Q-matrix can pass through split checking, graph construction, DAG audit, and leakage metrics. Computational cost is dominated by transition counting ($O(|\mathcal{F}_{\text{train}}|)$ per fold) and cycle pruning (polynomial in candidate edge count with a capped cycle enumerator), not by KT training itself. For million-scale logs (e.g. EdNet [8]), the same sparsity caps (K , top- k per node) bound $|E_{\text{pre}}|$ and keep audit tractable; distributed counting is an engineering extension not required for correctness.

Broader domains include corporate skills graphs (hierarchical job competencies), language-learning apps with prerequisite lexicon graphs, and MOOC platforms with forum or video-derived auxiliary relations. The similarity rule may need domain-specific co-occurrence (skills co-tested in the same certification item, linguistic collocation) when item-KC tags are single-label. Ground-truth cross-validation generalises whenever an expert DAG exists; otherwise behavioural inference remains auditable but uncalibrated. Cold-start strata extend to any taxonomy where tail concepts are pedagogically critical. DDR applies whenever a directed curriculum graph is used, including non-KT graph pretraining pipelines that import educational DAGs.

Implications for graph-enhanced KT. The protocol shows that KC graph construction should be treated as a first-class experimental component rather than an unexamined preprocessing step. Fold-specific edge inference, provenance logging, and DAG validation make it possible to identify leakage risks and graph construction choices that would otherwise remain hidden. The DDR results further show that generic graph augmentations can substantially disturb prerequisite structure, especially when they operate at the node level. This motivates follow-up work on prerequisite-preserving augmentation operators.

Cold-start reporting. The cold-start diagnostic shows why KT studies should not rely only on aggregate AUC. Rare knowledge components may behave differently from frequent knowledge components, and apparent gains or high scores on small strata may reflect class imbalance rather than genuine modelling improvement. Reporting per-stratum metrics gives reviewers a clearer view of whether a method is robust across the KC frequency distribution. Figure 3 links this

diagnostic to concrete graph topology: benchmarks with sparse E_{pre} (ASSISTments) still exhibit cold strata, whereas Junyi’s warmer strata coincide with denser arithmetic chains in the schematic plot.

Behavioural versus expert prerequisite curation. The ground-truth cross-validation on Junyi (Section 4.7) shows that train-only behavioural inference and expert prerequisite annotation are correlated at the node level but structurally distinct at the edge level. Two implications follow. First, behavioural inference cannot be treated as a low-cost approximation of expert curation, and papers that report graph-augmented KT results on behaviourally-inferred graphs should not claim those results transfer directly to settings with expert prerequisite graphs (or vice versa). Second, the two signals are complementary: in the most informative setup, a graph-augmented KT model would consume both an inferred behavioural graph and an expert prerequisite graph as separate relations, rather than treating either as a substitute for the other. We do not test this combined setting in the present paper, but the cross-validation provides the quantitative basis for proposing it.

Implications for graph-KT benchmarking. The empirical observation in Section 4.8 has direct implications for how the graph-KT subfield reports gains. If leakage-controlled evaluation changes the relative ranking of graph-augmented and sequence-only KT—and our three-fold ASSISTments and fold 0 XES3G5M pyKT diagnostics suggest it can—then benchmark tables that do not control for graph- construction provenance are not directly comparable across papers. We do not advocate dismissing reported gains in the graph-KT literature; we advocate *disaggregating* them. A reported AUC improvement of Δ on a benchmark should ideally be decomposed into three components: the part that survives leakage control, the part that depends on graph-construction choices (relation type, threshold, edge cap, encoder), and the part that depends on backbone strength. The protocol in this paper supplies the first component; the cold-start diagnostic supplies a partial view of the second; the third remains an open methodological question.

A practical recommendation follows. When introducing a new graph-augmented KT model, papers should report at minimum: (a) the fold-specific provenance of every retained graph edge; (b) the cold-start stratum on which the gain is most concentrated; (c) the sequence-only baseline matched in compute and hyperparameter budget; (d) when an expert prerequisite annotation is available, the ground-truth cross-validation of the consumed graph against that annotation, using the edge / direction / reachability / node-Jaccard metric family used in Section 4.7. Without these reports, an aggregate-AUC improvement is difficult to attribute. The protocol presented here makes (a), (b), and (d) tractable.

5.1 Threats to Validity and Metric Interpretation

Three experimental artefacts invite misreading if taken out of protocol context: the train-only versus full-log ablation (Section 4.6), aggressive DAG cycle pruning (Section 4.2), and elevated temporal-boundary violation rates in Table 3.

Ablation null shift does not negate leakage control (Issue 1). The controlled comparison in Table 13 moves headline pyKT AUC/ACC by less than 0.001 at three-decimal precision on ASSISTments, XES3G5M, and Junyi under our *deliberately leaky* full-log graph variant, while Synthetic C5 exhibits the largest released shift ($|\Delta\text{AUC}|\approx 0.006$ for GKT). We interpret this pattern as follows. First, *methodological hygiene stands independently of empirical impact*: graph statistics must not pool held-out learners regardless of whether a particular backbone exploits the shortcut. Second, the null shift on dense public benchmarks indicates that, under our support quantile, top- k cap, and global $K=5000$ filter, pooling additional learners before edge inference rarely changes which transitions survive—not that leakage is impossible in general. Third, train-only construction is *not penalising* headline metrics relative to this full-log variant in the released configuration; that is a positive outcome for protocol adoption rather than evidence that auditing is unnecessary. Synthetic corpora remain the most sensitive place to observe non-zero Δ in this release. Direct leakage metrics ($\text{ECR}_f^{\text{flag}}$, provenance logging) and ground-truth cross-validation (Section 4.7) therefore remain the primary audit deliverables; the ablation quantifies one specific pooling shortcut rather than certifying all graph-KT workflows.

Cycle pruning removes conflicts, not arbitrary mass (Issue 4). On Junyi fold 0, train-only temporal inference yields 2,152 directed candidates; lowest-confidence cycle pruning removes 1,013 edges ($\approx 47\%$) before 1,139 acyclic edges remain (Section 4.2). Johnson cycle enumeration finds 100 cyclic components in the raw candidate set; pruning breaks each cycle by dropping the *lowest-weight edge in that cycle* (`cycle\_prune\_lowest\_confidence` entries in `results/reports/*\_dag\_pruning\_log.csv`), a standard tie-break when behavioural co-occurrence induces bidirectional or conflicting precedence signals. Pruned edges therefore disproportionately reflect *conflict resolution* among spurious or weak transitions, not random deletion of the expert backbone: retained edges carry comparable or higher transition support (median weight ≈ 0.003 on Junyi fold 0 vs. ≈ 0.003 for pruned edges at the cycle tie-break). We do not claim optimality of this heuristic; sensitivity to pruning rate should be reported alongside DDR (Section 4.3), which shows modest structural disruption at moderate edge-drop rates (e.g. $\text{DDR}\approx 0.09$ at $p=0.10$ on Junyi).

TBVR measures within-train strictness, not split leakage (Issue 5). Table 3 reports TBVR_f up to ≈ 0.76 on XES3G5M and ≈ 0.73 on Synthetic C2. Under learner-disjoint folds, $\text{ECR}_f^{\text{flag}}=0$ confirms no cross-split structural leakage; TBVR_f instead operationalises Eq. (10) with an *indexed boundary inside the train fold*: for each learner, the earliest 70% of chronologically ordered train interactions define the audit cutoff, while prerequisite edges are inferred from transition evidence over the *full* train timeline. A post-cutoff train token is flagged when it participates in a retained edge transition straddling that boundary. High TBVR_f therefore signals *within-train temporal mixing* of edge evidence relative to the indexed audit boundary—not a valid/test split bug and not, by itself, a

violation of `train_only_flag` provenance. On XES3G5M, dense multi-skill co-assessment makes many late-train tokens support retained transitions; on Synthetic C2, the two-knowledge component toy graph retains a single directed edge after pruning, so most post-cutoff tokens inevitably touch that edge (a structural artefact of the sanity corpus). We treat elevated $TBVR_f$ as motivation to report temporal hygiene alongside ECR_f^{flag} , not as grounds to discard the pipeline.

Limitations. This version has six limitations. First, diagnostic KT baselines on ASSISTments use three-fold learner CV with bootstrap confidence intervals; Junyi, XES3G5M, and synthetic corpora remain fold 0 with collapsed intervals. Junyi includes cached native SKT but omits heavier graph-augmented pyKT rows (GKT, GIKT, DyGKT, DGEKT) because of memory and runtime limits. Table 12 populates ASSISTments three-fold Wilcoxon summaries; Junyi and XES3G5M rows await full multi-fold baseline exports. Second, the protocol’s similarity rule yields no E_{sim} on ASSISTments 2012. The audit (Section 4.4) confirms that this is a structural property of the dataset’s Q-matrix—100% of train items are single-skill, so item-overlap similarity is identically zero—rather than a pipeline failure. Replacing item-overlap with response-pattern correlation is a candidate extension. Third, the augmentation evaluation is limited to four label-agnostic families adopted from GraphCL [45]; structure-aware operators (e.g. prerequisite-preserving rewiring, attribute-only node drop, hierarchy-respecting subgraph sampling) are not yet covered by the DDR sweep. Fourth, ground-truth cross-validation (Section 4.7) is reported only on Junyi, because it is the only benchmark in our study with a public expert prerequisite annotation; extending the procedure to ASSISTments and XES3G5M requires constructing or obtaining expert annotations for those datasets, which is beyond the scope of this protocol paper. We also did not sweep the prerequisite-score threshold θ ; the reported numbers use $\theta = 0.5$ and retain the full Chang annotation. Fifth, calibration analysis and formal AUC comparison procedures are deferred to future work. Sixth, the train-only versus full-log graph ablation (Table 13) moves headline pyKT metrics by less than 0.001 AUC/ACC at three-decimal precision on ASSISTments, XES3G5M, and Junyi; Synthetic C5 shows shifts up to ≈ 0.006 AUC for GKT. We do not claim that other architectures, metrics, or inference rules would exhibit the same null shift.

6 Conclusion

We presented a leakage-controlled protocol for constructing and auditing concept graphs in KT. The protocol combines learner-based temporal splits, train-only graph construction, edge provenance logging, DAG cycle pruning, DDR augmentation diagnostics with four formally defined operators, ground-truth cross-validation against expert prerequisite annotation when available, and cold-start KC stratification. On Junyi Academy, ASSISTments 2012, and XES3G5M, the audit produces acyclic train-only prerequisite graphs after transparent cycle pruning. The DDR sweep separates the four augmentation families into four distinct structural regimes: $\text{DDR}(\text{attr_mask}) < \text{DDR}(\text{subgraph}) < \text{DDR}(\text{edge_drop}) <$

DDR(`node_drop`), consistent across all three datasets and all four perturbation strengths.

Ground-truth cross-validation on Junyi shows that train-only behavioural inference and expert prerequisite annotation are highly correlated at the node level (Jaccard ≈ 0.63) but only weakly correlated at the edge level (F1 ≈ 0.18 at $K = |\text{expert}|$, direction agreement ≈ 0.26). Qualitative disagreement patterns indicate that the two sources capture related but distinct aspects of prerequisite structure—behavioural co-occurrence versus pedagogical ordering—so behavioural inference should not be treated as a low-cost substitute for expert curation.

The cold-start diagnostic reveals per-stratum behaviour that aggregate metrics would otherwise hide (Table 6). A secondary empirical observation, made possible by the protocol itself, is that GKT pyKT checkpoints do not consistently outperform *simpleKT*/AKT under leakage-controlled evaluation on ASSISTments (three-fold mean) and XES3G5M (fold 0). We frame this not as a refutation of graph-augmented KT but as a motivation for leakage-controlled evaluation as a benchmarking precondition: if reported gains in the literature are partly attributable to graph-construction discipline rather than to structural information, then comparing graph-augmented and sequence-only KT requires the discipline this paper proposes.

The paper is deliberately positioned as a protocol/resource paper rather than a model paper. All scripts, configurations, logs, tables, figures, and prediction samples needed to reproduce the results are organised in the companion repository at https://github.com/tuanymc/p0_project.git, with source code under `src/`, configurations under `configs/`, pipeline scripts under `scripts/`, logs under `logs/`, and outputs under `results/`, including the ground-truth cross-validation artefacts under `results/gt_validation/junyi/`.

Ethics and data use. We use only public KT benchmarks under their respective licences. No personally identifiable information is used, and no claims about real learning outcomes are made.

References

1. Abdelrahman, G., Wang, Q., Nunes, B.P.: Knowledge tracing: A survey. *ACM Computing Surveys* **55**(11) (Feb 2023). <https://doi.org/10.1145/3569576>, <https://doi.org/10.1145/3569576>
2. Bai, Y., Liu, Z., Guo, T., Hou, M., Xiao, K.: Prerequisite relation learning: A survey and outlook. *ACM Computing Surveys* **57**(6) (May 2025). <https://doi.org/10.1145/3733593>, <https://doi.org/10.1145/3733593>
3. Chang, H.S., Kao, K.W., Wu, Y.T., Chen, H.T.: A crowdsourcing approach to building a K–12 prerequisite graph. In: *Proceedings of the Second ACM Conference on Learning @ Scale* (2015)
4. Chen, J., Liu, Z., Huang, S., Liu, Q., Luo, W.: Learning prerequisite structures from learning logs. In: *Proceedings of the International Conference on Artificial Intelligence in Education (AIED)* (2020)

5. Chen, J., Liu, Z., Huang, S., Liu, Q., Luo, W.: Improving knowledge tracing via pre-training question embeddings. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI) (2022)
6. Chen, J., Liu, Z., Huang, S., Liu, Q., Luo, W.: Hierarchical graph knowledge tracing. In: Proceedings of the ACM Web Conference (WWW) (2023)
7. Cheng, K., Peng, L., Wang, P., Ye, J., Sun, L., Du, B.: DyGKT: Dynamic graph learning for knowledge tracing. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 377–388. KDD '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3637528.3671773>, <https://doi.org/10.1145/3637528.3671773>
8. Choi, Y., Lee, Y., Shin, D., Cho, J., Park, S., Lee, S., Baek, J., Bae, C., Kim, B., Heo, J.: EdNet: A large-scale hierarchical dataset in education. In: International Conference on Artificial Intelligence in Education (AIED) (2020)
9. Corbett, A.T., Anderson, J.R.: Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* **4**, 253–278 (1995)
10. Cui, C., Yao, Y., Zhang, C., Ma, H., Ma, Y., Ren, Z., Zhang, C., Ko, J.: DGEKT: A dual graph ensemble learning method for knowledge tracing. *ACM Transactions on Information Systems* **42**(5) (May 2024). <https://doi.org/10.1145/3638350>, <https://doi.org/10.1145/3638350>
11. Feng, S., Cong, Y., Hou, J., Chen, Y., Ren, Z., Zhang, W., Liu, Q.: GraphMVP: Self-supervised multi-view pre-training for molecular representation learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2022)
12. Ghosh, A., Heffernan, N.T., Lan, A.S.: Context-aware attentive knowledge tracing. In: Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD) (2020)
13. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)
14. Hassani, K., Khasahmadi, A.H.: Contrastive multi-view representation learning on graphs. In: Proceedings of the 37th International Conference on Machine Learning (ICML) (2020)
15. Heffernan, N.T., Turner, T.E., Lourenco, A., Macasek, M., Nuzzo-Jones, G., Wildasinn, S.: The ASSISTments builder: Supporting rapid development and deployment of tutors that adapt to the individual. In: Proceedings of the 8th International Conference on Intelligent Tutoring Systems (2006)
16. Huang, Z., Liu, Q., Chen, E., Wu, H., Su, Y., Liu, Z., Ma, H.: Learning or forgetting? a dynamic approach for tracking the knowledge proficiency of students. In: Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM) (2022)
17. Johnson, D.B.: Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing* **4**(1), 77–84 (1975). <https://doi.org/10.1137/0204007>
18. Junyi Academy: Junyi academy online learning activity dataset. PSLC DataShop (2015), <https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198>, available at PSLC DataShop
19. Khajah, M., Wing, R.M., Lindsey, R.V., Mozer, M.C.: Incorporating latent factors into knowledge tracing to predict individual differences in learning. In: Proceedings of the 9th International Conference on Educational Data Mining (EDM) (2016)
20. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)* (2017)

21. Laguna, Á., Martínez-Abad, F., Núñez, J.C., Pérez-Sanagustín, M., Sánchez-Elez, M.: Mining and visualizing learning paths to support self-regulated learning. *IEEE Transactions on Learning Technologies* **11**(4), 498–511 (2018)
22. Lee, W., Chun, J., Lee, Y., Park, K., Park, S.: CL4KT: Contrastive learning for knowledge tracing. In: *Proceedings of the ACM Web Conference (WWW)* (2022)
23. Lin, Y.C., Liu, Q., Wu, H., Chen, E., Su, Y., Chen, Z.: Discovering prerequisite relations among concepts in MOOCs. In: *Proceedings of the 25th ACM International Conference on Information and Knowledge Management (CIKM)* (2016)
24. Liu, Q., Tong, S., Liu, C., Huang, W., Chen, E.: Exploiting cognitive structure for adaptive learning. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (2021)
25. Liu, Z., Liu, Q., Chen, J., Huang, S., Luo, W.: simpleKT: A simple but tough-to-beat baseline for knowledge tracing. In: *International Conference on Learning Representations (ICLR)* (2023)
26. Liu, Z., Liu, Q., Chen, J., Huang, S., Tang, J., Luo, W.: pyKT: A Python library to benchmark deep learning based knowledge tracing models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems*. vol. 35, pp. 13248–13261. Curran Associates, Inc. (2022), https://proceedings.neurips.cc/paper_files/paper/2022/hash/558778f30960570a241476f4e648c66e-Abstract-Datasets_and_Benchmarks.html
27. Liu, Z., Liu, Q., Guo, T., Chen, J., Huang, S., Zhao, X., Tang, J., Luo, W., Weng, J.: XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*. vol. 36, pp. 40643–40662 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/67fc628f17c2ad53621fb961c6bafcaf-Abstract-Datasets_and_Benchmarks.html
28. Nakagawa, H., Iwasawa, Y., Matsuo, Y.: Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In: *IEEE/WIC/ACM International Conference on Web Intelligence (WI)* (2019)
29. Pan, L., Zhong, Y., Chen, E., Liu, Q.: Learning prerequisite relations from educational data. In: *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)* (2020)
30. Pandey, S., Karypis, G.: A self-attentive model for knowledge tracing. In: *Proceedings of the 12th International Conference on Educational Data Mining (EDM)* (2019)
31. Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L.J., Sohl-Dickstein, J.: Deep knowledge tracing. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2015)
32. Schein, A.I., Popescul, A., Ungar, L.H., Pennock, D.M.: Methods and metrics for cold-start recommendations. *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (2002)
33. Shen, S., Liu, Q., Chen, E., Wu, H., Huang, Z., Zhao, W., Ma, H., Wang, H., Yu, S., Gao, W.: Convolutional knowledge tracing: Modeling individualization in student learning process. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
34. Song, X., Li, J., Lei, Q., Zhao, W., Chen, Y., Mian, A.: Bi-CLKT: Bi-graph contrastive learning based knowledge tracing. *Knowledge-Based Systems* **241**, 108274 (2022). <https://doi.org/10.1016/j.knsys.2022.108274>

35. Thakoor, S., Tallec, C., Azar, M.G., Munos, R., Veličković, P., Valko, M.: Bootstrapped representation learning on graphs. arXiv preprint arXiv:2102.06514 (2021)
36. Tong, S., Liu, Q., Huang, W., Liu, Z., Chen, E., et al.: Structure-based knowledge tracing: An influence propagation view. In: IEEE International Conference on Data Mining (ICDM) (2020)
37. Veličković, P., Fedus, W., Hamilton, W.L., Liò, P., Bengio, Y., Hjelm, R.D.: Deep graph infomax. In: International Conference on Learning Representations (ICLR) (2019)
38. Wan, Y., Du, B., Ye, J., Sun, L., Peng, L.: Contrastive learning for knowledge tracing. In: Proceedings of the ACM Conference on Knowledge Discovery and Data Mining (KDD) (2021)
39. Wang, S., Lin, J., Huang, S.: Corekt: A compact and effective knowledge tracing model. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM) (2023)
40. Wang, S., Lin, J., Huang, S., Wang, J., Wu, J.: A dual-encoder contrastive learning model for knowledge tracing. *Entropy* **27**(7), 685 (2025). <https://doi.org/10.3390/e27070685>, <https://www.mdpi.com/1099-4300/27/7/685>
41. Wang, S., Lin, J., Huang, S., Wu, J.: Grapkt: Graph-based knowledge tracing for adaptive learning. In: Proceedings of the IEEE International Conference on Data Mining (ICDM) (2022)
42. Xia, J., Wu, L., Chen, J., Hu, B., Li, S.Z.: SimGRACE: A simple framework for graph contrastive learning without data augmentation. In: Proceedings of the ACM Web Conference (WWW) (2022)
43. Xiong, X., Zhao, S., Huang, C., Sun, W., Chen, E.: Graph-enhanced context-aware knowledge tracing. In: Proceedings of the ACM Web Conference (WWW) (2023)
44. Yang, Y., Shen, J., Qu, Y., Liu, Y., Wang, K., Zhu, Y., Zhang, W., Yu, Y.: GIKT: A graph-based interaction model for knowledge tracing. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML-PKDD) (2021)
45. You, Y., Chen, T., Sui, Y., Chen, T., Wang, Z., Shen, Y.: Graph contrastive learning with augmentations. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)
46. Yudelson, M.V., Koedinger, K.R., Gordon, G.J.: Individualized bayesian knowledge tracing models. *Artificial Intelligence in Education* (2013)
47. Zhang, J., Shi, X., King, I., Yeung, D.Y.: Dynamic key-value memory networks for knowledge tracing. In: Proceedings of the 26th International Conference on World Wide Web (WWW) (2017)
48. Zhu, Y., Xu, Y., Liu, Q., Wu, S., Wang, L.: An empirical study of graph contrastive learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
49. Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., Wang, L.: Graph contrastive learning with adaptive augmentation. In: Proceedings of the Web Conference (WWW) (2021)